

2. 氛围编程的关键实践 (2)

— Agentic IDE基于项目的上下文管理

大纲

01 项目的基本概念

项目是围绕一个可交付目标组织起来的工作单元。

- 项目定义
- 项目作为Agentic IDE 的上下文容器
- Demo：构建项目

02 版本管理基础

Git版本管理的核心概念与实践

- 版本控制：项目的时间机器
- Vibe Coding中的版本管理
- Demo 与最佳实践

03 基于项目的上下文管理

探索Agentic IDE的基于项目的上下文管理方式。

- 基于项目的上下文管理的五种方式
  - 会话上下文
  - 自动上下文
  - 手动上下文
  - 规则上下文
  - 运行上下文
- Demo 与最佳实践

什么是项目

项目是围绕一个可交付目标组织起来的工作单元

项目定义

面向目标

每个项目都服务一个**明确目标**，如完成课程案例、实现功能模块、交付一次作业。

目标是项目的驱动力，定义了工作的终点和验收标准。

面向过程

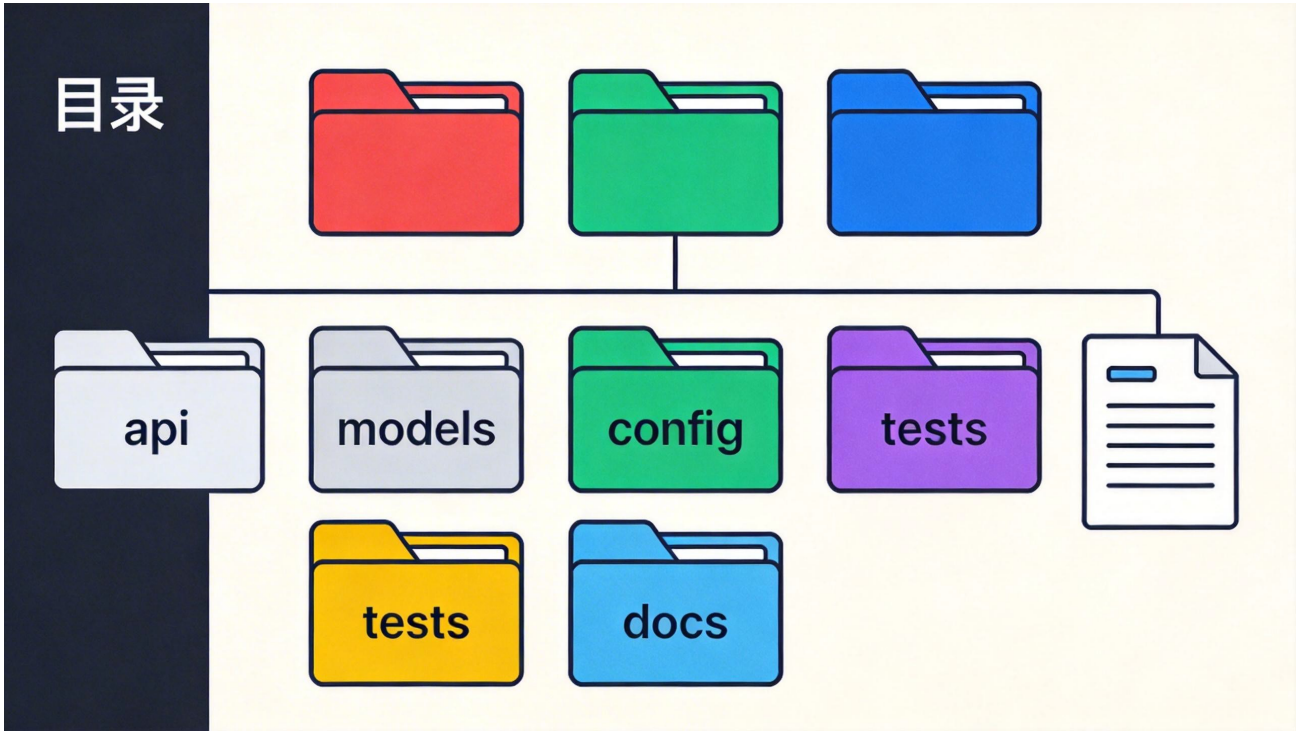
项目同时记录从**需求到实现、测试、迭代**的全过程，便于追踪与复用。

过程是项目的轨迹，承载着经验积累和知识沉淀。

面向协作

项目为人和智能助手提供**共同上下文**，让"**理解–执行–验证**"形成闭环。

协作是项目的灵魂，实现人机协同的高效工作模式。



核心定义：在本课程里，项目是围绕一个可交付目标组织起来的工作单元。

它不仅包含代码，还包含**资源、配置、文档**，以及与智能助手协作过程中沉淀的**规则、任务和会话过程**。

项目与聊天模式的区别

聊天模式

无状态对话

单次对话通常是"无状态"的，每次对话都是独立的，缺乏连续性。

无法处理复杂依赖

难以处理复杂的多文件依赖（导入/导出），无法理解模块间的关系。

缺乏全局视野

难以理解整体架构，只能看到孤立的代码片段。

项目工作方式

持续性工作

应用的构建通常是持续性的项目工作，支持长期迭代和演进。

完整软件构成

应用软件不仅仅是代码片段，软件 = 代码 + 资源 + 配置 + 文档。

结构化组织

它们必须以某种形式组织在一起，形成有机的整体。

项目工作方式让智能助手能够理解完整的软件构成，而不仅仅是孤立的代码片段。

文件夹作为项目的工作目录（1）

物理与逻辑

文件夹

操作系统的物理存储单位。
是文件的容器。

项目

软件开发的逻辑组织单位。它
是有目标、有结构的代码集合。

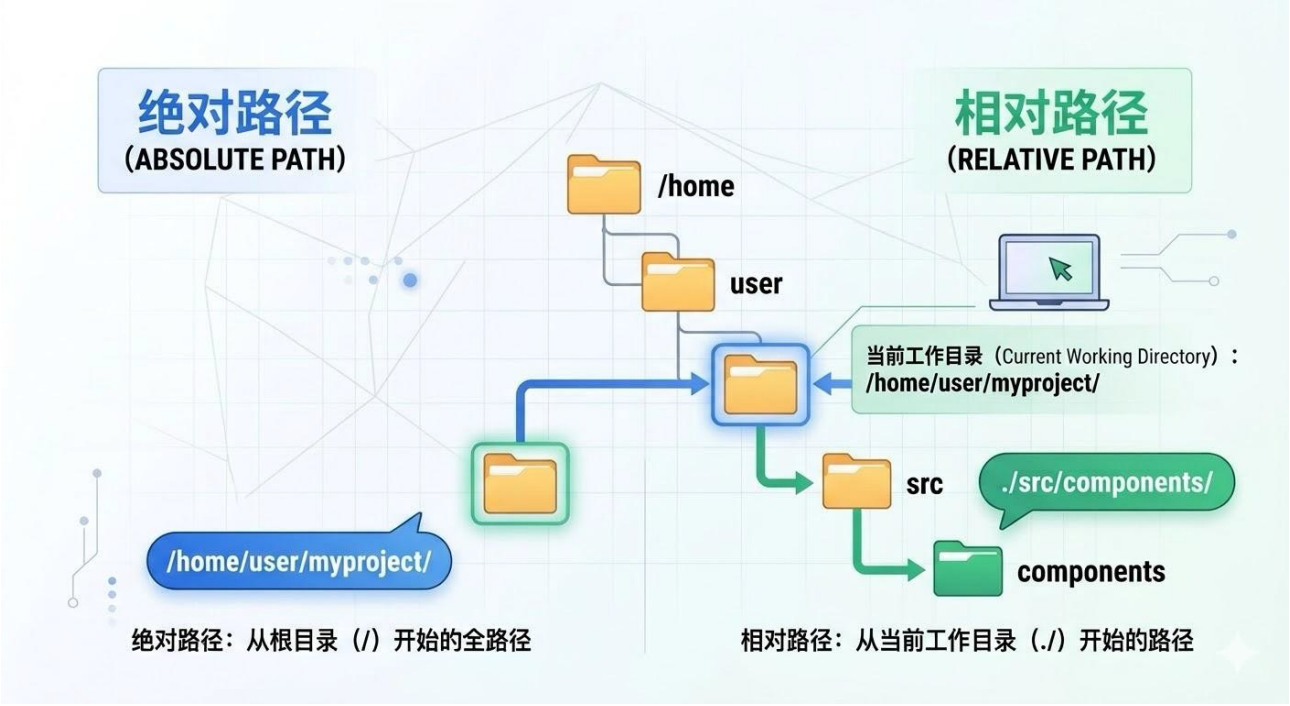
路径

智能助手需要准确的路径来定位文件

绝对路径: /home/user/project/

相对路径: ./src/components/

文件系统



文件夹作为项目的工作目录（2）

作为项目的文件夹

代码和文档以外，还要包含以下内容：

元数据

添加描述文件（如 package.json, requirements.txt）

版本控制

初始化 .git 目录

开发环境配置

添加 .trae, .vscode 等配置目录

当你用**开发环境**打开一个文件夹时，它就不仅仅是文件夹了，它被"**升级**"为一个项目。

氛围编程的起点：构建一个项目

起一个恰当的名称

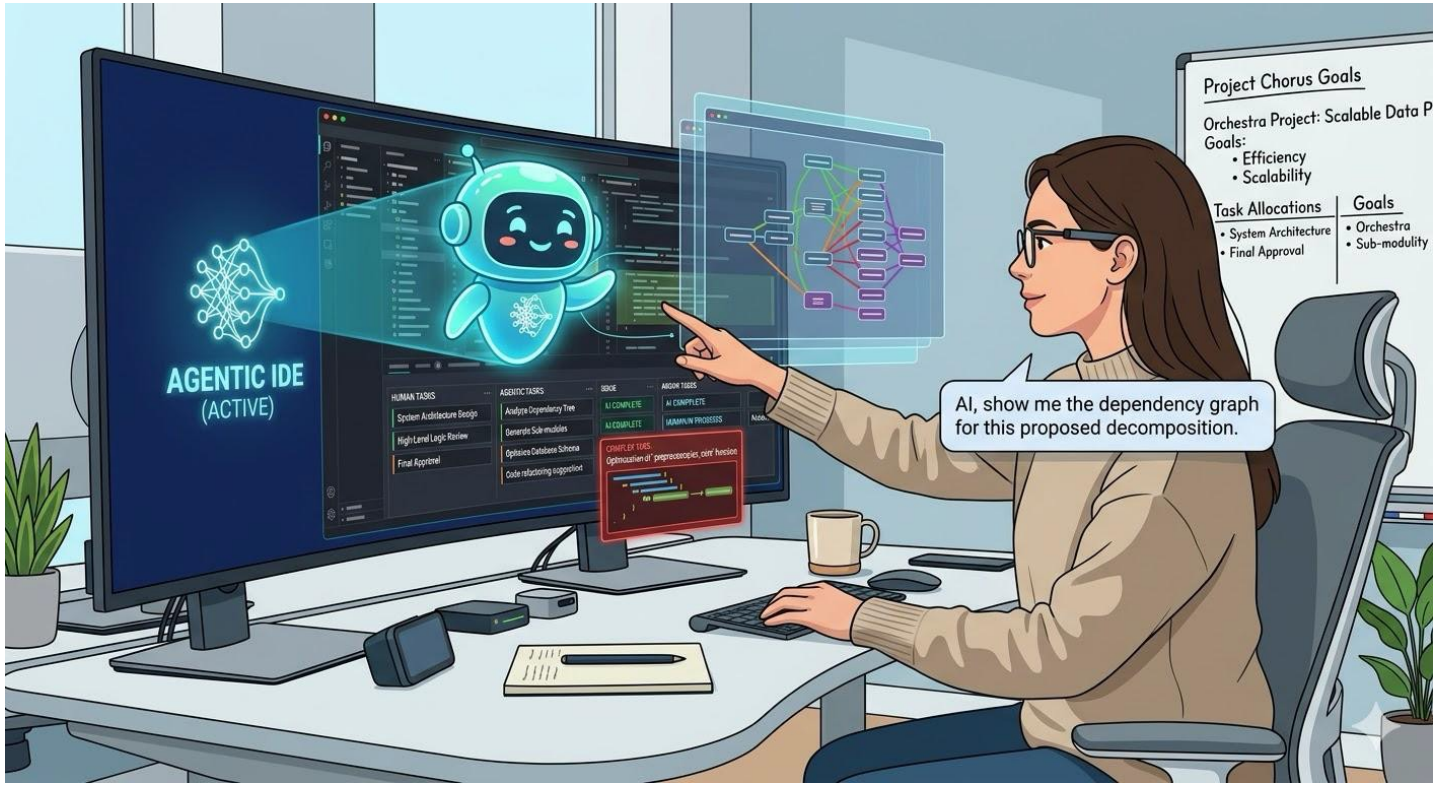
项目名称应简洁明了，反映项目目标和内容

设定工作根目录

将开发环境的工作目录设定为这个文件夹

开始对话

开始与开发环境对话，描述你的需求和目标



开始氛围编程：与智能体开发环境的每一次对话中，智能体开发环境都会尝试获取回答或执行问题所需的相关信息与上下文。

氛围编程的起点：构建一个项目

Demo



项目作为Agentic IDE的上下文容器

智能体开发环境的视角

Agentic IDE(智能体开发环境)， 它有点像你请来的一个智能合作伙伴

共享项目信息

理解项目结构

肩并肩协作

01 边界控制

出于安全性需求，项目定义了智能助手能"看到"和"操作"的范围。

限定AI的作用域，防止AI访问无关文件

02 上下文管理

开发环境会自动扫描项目内的所有文件并建立索引，最大程度发挥其能力。

智能自动检索相关文件

03 过程和结果保存

保存项目成果，以及与智能体开发环境的对话历史、项目规则等。

持久化存储交互记录，可追溯协作记录

核心功能： 项目作为容器，一方面限定了智能助手的边界，另一方面提供了上下文，同时也保留了人与智能体开发环境交互的过程记录。

项目的延伸

这种工作方式不仅限于写代码，生活、学习和工作中的复杂任务同样适用。

写书/论文

- ✓ 章节分文件管理
- ✓ 参考文献集中存放
- ✓ 版本控制追踪修改

✓ 结构化写作，便于版本控制

旅游规划

- ✓ 行程单统一管理
- ✓ 攻略文档整理
- ✓ 预订凭证归档

✓ 信息集中，随时查阅

课程学习

- ✓ 课件有序组织
- ✓ 作业分类管理
- ✓ 笔记集中存放

✓ 知识管理，高效复习

项目管理 | **家庭事务** | **健康管理** | **创意设计**

版本管理基础

版本控制：项目的时间机器

定义

一种 记录文件内容变化 ，以便将来查阅特定版本修订情况的系统。

核心功能

历史回溯

随时将文件回退到之前的状态，像时间机器一样穿越到任意历史节点

并行开发

在不影响主线的情况下进行新功能开发或实验，多条时间线并行演进

协同工作

多人同时修改同一项目而不会发生混乱，自动合并与冲突解决



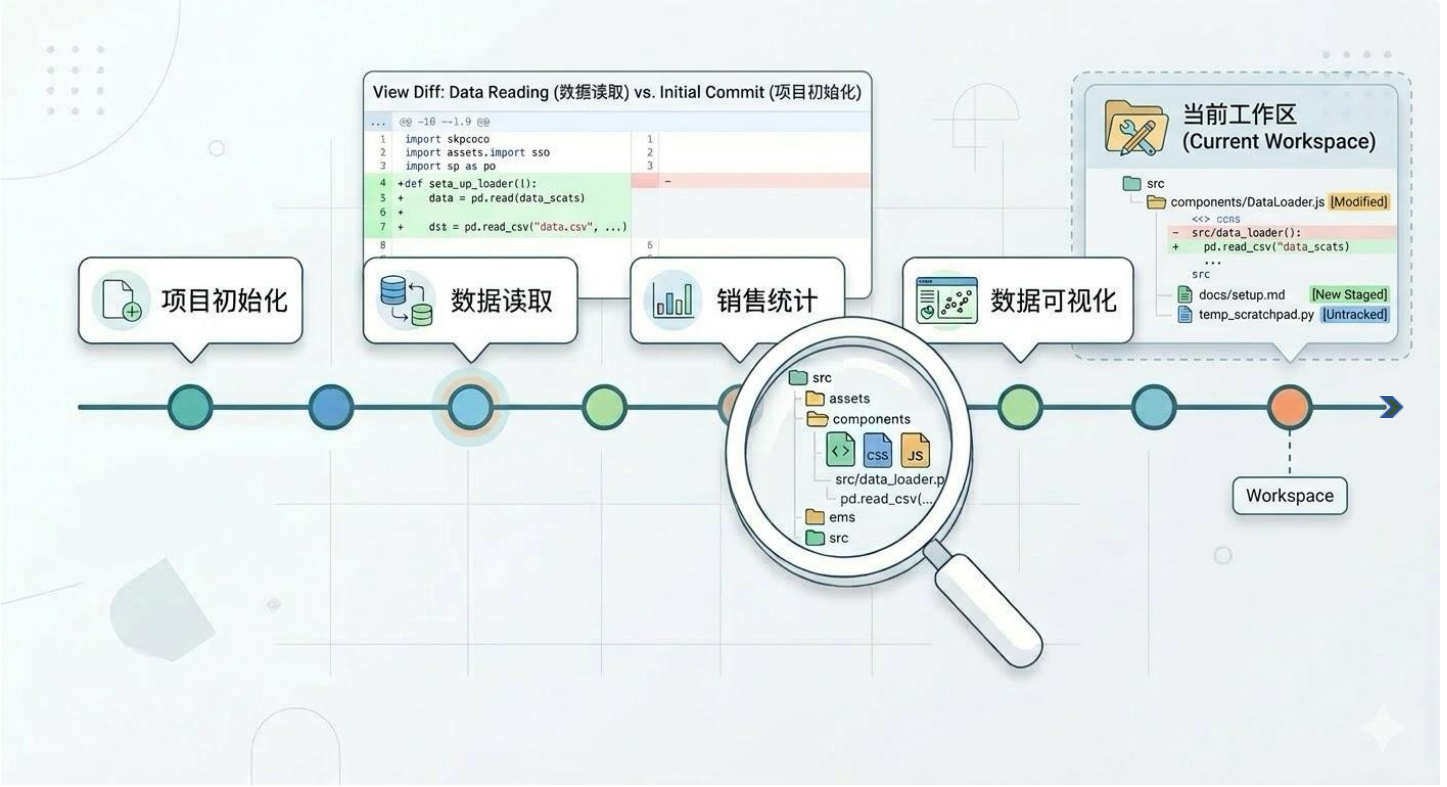
主流工具：Git — 最流行的分布式版本控制系统

版本控制：项目的时间机器

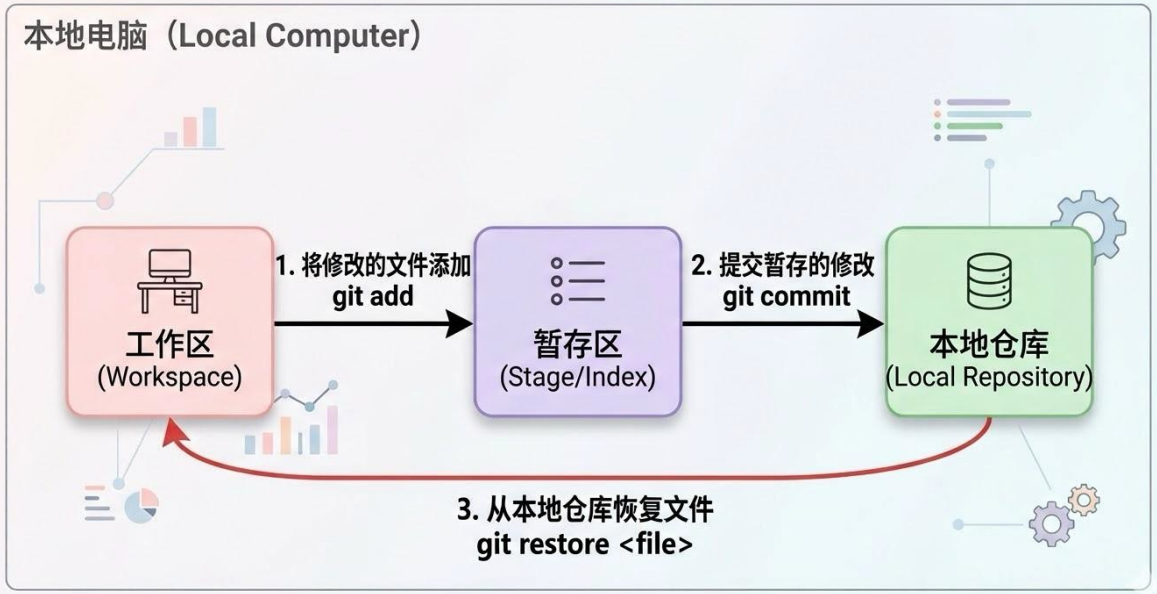
Demo



项目全生命周期管理



Git的三层模型(仅本地版本管理)



工作区 Workspace

你正在编辑的文件，是代码的实际工作场所

暂存区 Staging Area

通过 git add 选中的"本次要提交内容"

本地仓库 Local Repository

通过 git commit 形成的历史快照

- **直观理解：** `git add` 是"选择要进快照的改动"， `git commit` 是"把这些改动固化为一个版本节点"

- 扩展学习：远程仓库（github,gittee）

如何与版本管理工具交互



自然语言交互

通过AI助手或智能工具以对话形式完成Git操作，**无需记忆复杂命令语法**。



图形界面交互

借助IDE内置工具等可视化软件进行操作。适合或偏好直观拖拽操作的用户，提供清晰的提交历史图谱展示。



命令行交互

直接在终端输入Git指令完成操作，是最高效灵活的方式。适合自动化脚本编写、服务器环境操作，或追求极致效率的专业开发者，**支持完整的Git功能集**。

02

Vibe Coding 与版本管理

从备份工具到智能助手的上下文提供者，探索Agentic IDE的版本管理新范式

为什么 Vibe Coding 更需要版本管理？

高频变更

AI 可以在 几秒钟内生成大量代码 。
如果没有版本控制，错误的生成瞬间就会覆盖掉正确的逻辑，且难以凭记忆撤销。

实验性质

我们经常需要让 AI "试一试"。
比如："把这个函数改成异步的"。这种探索性的修改需要极低成本的回滚机制。

> AI 的加入改变了代码生产的速度和性质，在 Vibe Coding 中我们开发更加倾向于高频变更和实验性质。
>
> 因此，**版本管理就显得更为重要**。

版本管理作为 AI 的上下文

Vibe Coding 中，版本管理不仅是备份代码、充当时间机器， 还能为 AI 提供特定信息的上下文

智能代码审查

AI 会对比当前分支与 main 分支的差异。

上下文 = 功能分支 - 主分支

AI 重点审查新增的代码行

通过版本差异，AI 能够精准定位需要审查的内容

"由于我的修改导致报错"

当你问"我刚才改了什么导致这个错？"时，Agent 会主动调用：

```
git diff HEAD
git log -p -1
```

上下文 = 最近的变更记录。AI 通过回溯历史来定位引入错误的源头

通过版本管理，我们为项目建立了一个 **可回溯的时间轴** ，使得 AI 的每一次"生成"都能获得更多相关的上下文信息。

Vibe Coding的案例：咖啡店销售数据自动化简报

我们通过一个实际案例——咖啡店销售数据自动化简报，来展示 Git 如何在 Vibe Coding 流程中发挥作用。

案例概览：三个阶段逐步构建



数据读取模块

负责读取 Excel 数据





统计分析模块

负责计算核心指标





可视化模块

负责生成图表

Vibe Coding的案例：咖啡店销售数据自动化简报（2）

Demo



Vibe Coding的案例：咖啡店销售数据自动化简报（3）

项目的时间维度

Session 1 数据读取

Session 2 统计分析

Session 3 可视化

Session 4 优化迭代

图形 自动

feat: 添加销售数据可视化报表功能 tiange zhang main

.gitignore M
data_loader.py src M
visualizer.py src A
feat: 添加销售统计模块，计算每日销售统计(总收入，销售额冠...
stats.py src A
test_stats.py tests A
feat: 实现销售数据读取与清洗模块 tiange zhang
data_loader.py src A
test_data_loader.py tests A
项目初始化 tiange zhang
.gitignore A
README.md A
requirements.txt A
__init__.py src A
__init__.py tests A
sales_january.csv tests/raw_data A
sales_january.xlsx tests/raw_data A

项目的会话过程

今天

生成每日销售额柱状图

创建统计汇总模块

读取 Excel 销售数据

初始化Python项目并配置Git

文件内容的变更历史

```
146 .AppleDouble
147 .LSOverride
148
```

```
146 .AppleDouble
147 .LSOverride
148+ tests/temp/daily_report.png
149
```

Vibe Coding中的版本管理的核心价值

通过版本管理工具，我们能更加有条不紊地管理和演化项目，将项目的**整个生命周期可视化**：

存档点

每完成一个功能模块（一个会话任务），就进行一次提交。

- 清晰的里程碑标记

心理安全感

因为有"存档"，我们可以放心地让 AI 进行大胆的代码重构或功能添加。

- 不用担心破坏已有功能

清晰的演进脉络

相比于堆砌代码，Git 提交记录清晰地展示了项目是如何一步步"生长"出来的。

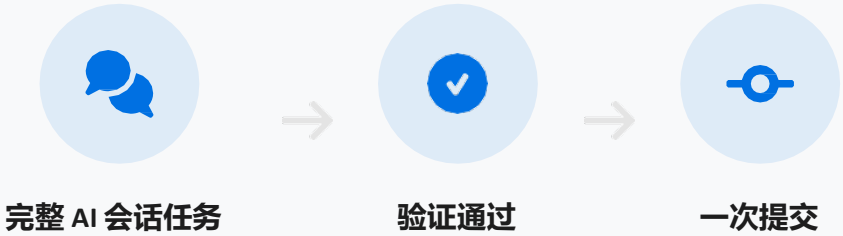
- 可追溯的项目演进史

> 版本管理让 AI 辅助开发从**"无序生成"**变成**"有序演进"**

版本管理最佳实践：会话驱动提交

会话驱动提交工作流

在 Vibe Coding 模式下，我们建议采用 会话驱动提交 的工作流。



核心原则

- **小步快跑：** 利用 Agentic IDE 快速迭代
- **及时存档：** 功能一旦跑通，立即提交
- **描述意图：** 记录"为什么需要这次修改"，而不仅仅是"怎么做的"

提交说明的撰写

不仅仅描述"改了什么代码"，更应**包含提示词和上下文信息**，以便回溯 AI 的生成逻辑。

反例

```
fix bug
```

过于简单，无法追溯意图

正例

```
fix: 修复数据解析错误（处理空值情况）
```

清晰描述问题与解决方案

提示： 提交说明是写给**未来的自己和AI**看的，请包含足够上下文信息

版本管理最佳实践： Diff变更审查

提交前先看改动差异，把**"代码生成正确"**变成**"变更可解释、可控、可回退"**

建议重点关注三类风险：

误删风险

检查是否删掉了已有可用逻辑、边界处理或异常处理分支。

⚠ AI 可能会"过度清理"代码

命名不当

检查变量名、函数名、字段名是否与上下文一致，避免前后语义错位。

⚠ 命名不一致会导致理解困难

重复逻辑

检查是否引入了功能重复但实现不同的代码块，避免后续维护分叉。

⚠ DRY 原则：Don't Repeat Yourself

```
#查看工作区与暂存区的差异
git diff
```

```
#查看暂存区与仓库的差异
git diff —staged
```

版本管理最佳实践：.gitignore 配置

.gitignore 的作用

避免**临时文件、缓存文件、构建产物**被误提交，降低 AI 高频生成带来的仓库噪声。

将**不需要纳入版本管理**的文件添加到 `.gitignore` 中

为什么要配置 .gitignore

- 保持仓库**干净**，只包含有价值的代码
- 减少**仓库体积**，提高克隆速度
- 避免**敏感信息**（如 API 密钥）泄露
- 降低 AI 生成代码带来的**噪声**

常见 .gitignore 配置示例

```
# Python
__pycache__/
*.pyc
*.pyo
*.pyd
.Python *.so
# 虚拟环境
venv/
.....
```

提示： 不同技术栈有不同的 .gitignore 模板，可以在 github/gitignore 找到官方推荐配置

基础命令

1 初始化工作区

```
#进入项目目录
cd my-project #
初始化仓库  git
init
```

效果： 生成隐藏的 .git 目录，存放所有历史记录

2 提交到暂存区

```
#添加指定文件
git add filename.py
#添加当前目录下所有修改
git add
```

效果： 把文件放入"待提交清单"，确认AI生成的代码没问题

3 提交到仓库

```
#提交并附带说明
git commit -m "feat: 完成数据读取功能"
```

隐喻： 将"待提交清单"内容正式归档，生成永久快照

⚠ 提交备注(commit message) 的重要性

- **反例：**
  fix bug
- **正例：**
  fix: 修复数据解析错误（处理空值情况）
- **Commit Message** 是写给**未来的自己和AI**看的，
  请务必清晰描述**意图**

更多命令：查看历史与回滚

查看提交历史

```
#查看详细历史
git log
```

```
查看精简历史（一行一条）
git log -oneline
```

查看项目的**时间轴**，了解每一次变更的完整记录

丢弃未提交修改

```
#恢复工作区文件（推荐）
git restore .

#旧写法（仍可用）
git checkout .
```

当你只想撤销**还没提交**的本地改动时，使用这组命令最合适

在 Vibe Coding 中的应用

快速迭代

AI生成代码后，通过 `git diff` 查看变更，快速验证是否符合预期

快速回滚

当AI生成的代码不符合预期时，使用 `git restore` 一键撤销，回到干净状态

变更追溯

通过 `git log -p` 查看每次提交的详细变更，理解代码演进过程

> 版本控制让AI的每一次"生成"都能被**追踪、验证和回滚**

更多命令：分支与合并

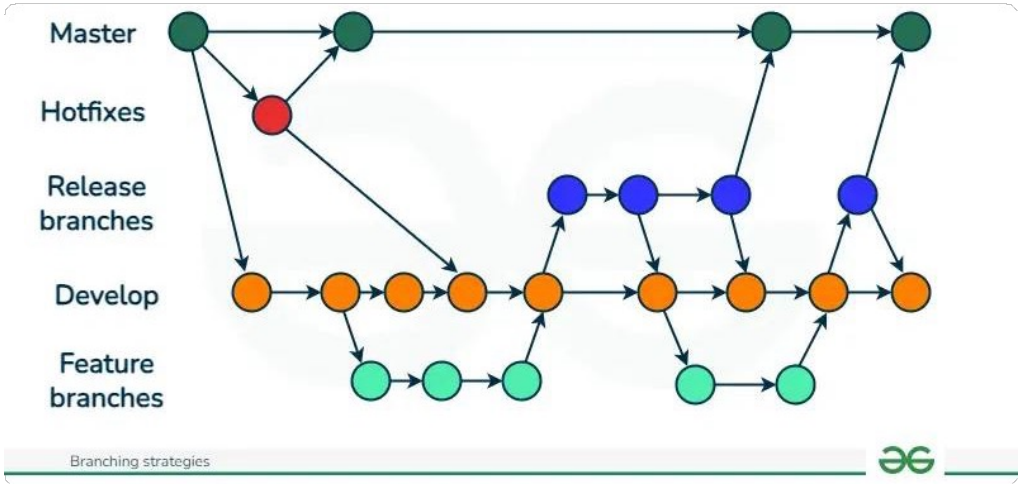
分支 Branch

创建一个**独立的开发副本**，在这个隔离的环境中自由实验，完全不影响主分支的稳定性。

```
#创建并切换到新分支
git switch -c feature-new-ui
```

场景："AI，我想尝试将界面改为深色模式，但希望保留当前可运行的版本以防万一。"

→ 先开个 dark-mode 分支再让 AI 改



合并 Merge

当在分支中的开发经过验证后，可以将其**合并回主分支**。

```
#先切回主分支
 git switch main
#把 feature 分支的成果合并过来
 git merge feature-new-ui
```

场景："深色模式功能验证通过，准备集成到主版本。"

→ Merge 回 Main 分支

在 Vibe Coding 中的价值

- 让AI进行**大幅度重构**或**实验性修改**时，完全不影响主分支稳定性
- 可以**并行尝试多种方案**，对比后再决定采用哪个分支
- 实验失败时，**直接删除分支**即可，无需清理代码

基于项目的上下文管理

探索Agentic IDE提供的基于项目的上下文管理方式

基于项目的上下文管理方式

Agentic IDE 智能体开发环境提供了以下几种管理上下文的方式：

1. 会话上下文
Session Context

2. 自动上下文
Auto Context

3. 手动上下文
Manual Context

4. 规则上下文
Rule Context

5. 运行时上下文
Runtime Context

这五种上下文方式构成了项目的**上下文管理**，各有优势与适用场景

1. 会话： 会话即微型项目（1）

定义与作用

1 定义

智能体开发环境与用户之间的每次对话，都被视为一个会话，这与聊天模式会话是同一概念，只是出现在Agentic IDE的开发环境的会话窗口中。

2 作用

- ✓ 记录对话历史，方便后续参考和复用。
- ✓ 提供上下文，帮助智能助手理解用户的意图和需求。

3 持久化

默认开启，会话结束后上下文会被保存，可以查询历史会话记录。

1.会话： 会话即微型项目（2）

★ 最佳实践

原子化会话

一个会话只专注解决一个明确的任务（如"修复登录缺陷"或"重构支付模块"）

✓ 保持上下文纯净

会话即版本

利用开发环境的历史记录功能，将每个会话视为一次提交。

✓ 失败可废弃会话

善用回滚

在长会话中，如果智能助手跑偏了，不要试图通过更多对话去"纠正"它。

✓ 直接回滚到之前Prompt

1. 会话： 会话即微型项目（3）

⚠ 反例提醒

错误做法： 把"修复登录缺陷、重构支付流程、补课程作业说明"放进同一个会话

→ **后果：** 上下文相互污染，智能助手频繁切换目标，输出质量明显下降

→ **正确做法：** 按任务拆分为三个独立会话，每个会话只保留当前任务所需信息

2. 自动上下文： 智能匹配相关文件（1）

什么是自动上下文?

- 开发环境自动扫描、索引整个项目中的所有文件
- 使用检索增强生成技术，根据你的问题智能匹配相关文件
- 无需手动指定，智能助手"猜"你需要什么

自动上下文包含什么?

- 当前打开的文件内容
- 光标所在位置的代码
- 项目的文件结构和依赖关系
- 编辑器中的报错和警告信息

使用场景

你问：

"解释一下用户认证模块是如何工作的？"

智能助手自动：

检测到 auth.py 中的代码，自动关联相关的 user_model.py 和 config.py，综合解释认证流程。

2. 自动上下文： 智能匹配相关文件（2）

局限性

延迟

对超大项目，首次索引或文件变更后的重索引可能需要时间

误判

智能助手可能"猜错"，特别是在大型项目中

上下文消耗

可能需要耗费更多上下文长度

依赖性

对于隐式依赖（如运行时配置）理解有限

应对策略： 当出现连续多轮会话仍无法收敛，应**切换为手动指定上下文**并缩小范围

3. 手动上下文：精准控制注意力（1）

什么是手动上下文?

- 你主动告诉智能助手应该关注哪些文件或代码
- 通过显式引用，确保智能助手看到正确的信息
- 精准控制智能助手的"注意力"

手动引用的方式

引用文件 语法：@文件名

让智能助手查看特定文件

引用文件夹 语法：@文件夹名/

让智能助手了解整个目录结构

选中代码 直接选中

针对特定代码片段提问

粘贴内容 直接粘贴

分享外部信息

3. 手动上下文：精准控制注意力（2）

★最佳实践

1 新建会话时引用

先引用与当前任务相关的文件或文件夹

2 遇到报错时引用

引用错误文件和报错信息

3 最小必要引用原则

优先引用直接相关的 2-4 个文件，再按需要逐步补充

4 先核心后依赖

先给核心文件，再给依赖文件，避免一次性引用过多内容造成噪声

</> 示例

"请根据 @api_v2_migration.md 中的新接口定义， 重构 @payment_service.py 中的支付逻辑，注意保留 @legacy_utils.py 中的兼容 性处理"

清晰指定了三个关键文件及其作用

4. 规则上下文：项目级知识（1）

什么是规则上下文?

- 通过配置文件定义的项目级规则
- 持久化存储，每次对话自动生效
- 定义代码风格、技术栈、沟通方式等

规则文件的类型与创建

用户规则

全局

文件位置：开发环境设置中
作用范围：所有项目（个人偏好）
创建方式：在开发环境中的规则页面直接编辑

项目规则

项目级

文件位置：.trae/rules 或 .cursorrules
作用范围：当前项目
创建方式：在项目根目录新建文件
(通常是纯文本或 Markdown)

4. 规则上下文：项目级知识（2）

规则的价值

减少重复说明

不需要每次对话都交代偏好

保持一致性

团队协作时统一标准

降低沟通成本

智能助手自动按规则行事

规则应该包含什么？

```
# 项目规则示例

## 沟通方式
- 生成界面上的文字使用中文
- 代码中的命名和注释使用英文
- 解释技术概念时使用通俗语言

## 代码规范
- Python：遵循 PEP8，4空格缩进
- 所有函数必须包含 docstring
- 变量使用 snake_case 命名

## 技术栈
- 后端：Python + FastAPI
- 前端：HTML + TailwindCSS

## 开发流程
- 先更新 Todo，再写代码
- 每个任务完成后提交 Git
```

4. 规则上下文：项目级知识（3）

规则失效/过时的处理策略

预先定义的通用规则无法覆盖特殊场景：

```
## 项目规则

要求“所有页面文案必须中文”

## 本次任务

接入第三方支付页，按钮文案必须保持
英文原文（如 Pay Now）。
```

处理策略

临时覆盖并声明范围

在当前会话明确“本次任务临时规则”与生效范围，避免误扩散。

回写项目规则文件

将已验证的新规则更新到项目规则中，删除失效条目。

补充变更说明

记录“为什么改、改了什么、何时生效”，便于后续复盘与协作。

运行时上下文：代码的实际表现（1）

什么是运行时上下文？

- 代码执行过程中产生的动态信息
- 包括报错、日志、测试结果等
- 让智能助手能够"看到"代码的实际表现

运行时信息的类型

类型	来源	用途
代码评审或语法错误	开发环境静态分析	代码规范检查
运行报错	终端输出	调试和修复
测试结果	测试框架	验证正确性
日志输出	程序运行	追踪执行流程

运行时上下文：代码的实际表现（2）

最佳实践：自动修复循环

智能体开发环境的核心能力： 开发环境可以自动运行命令 → 捕获报错 → 自动修复 → 再次运行



这比手动复制粘贴要**高效得多**

提供充分信息： 需要手工提供时，完整复制错误信息（包括堆栈跟踪）或者提供图片。

明确上下文： 告诉智能助手"这是运行后的报错"，让它理解这是结果而非代码。

示例对话

你：

运行 `@app.py` 后出现以下报错：

```
Traceback (most recent call last):
File "app.py", line 23, in
conn = create_connection()
Error: Database connection failed
```

智能助手：

根据报错，问题出在第23行的数据库连接配置。建议检查环境变量是否正确设置...

基于项目的上下文

Demo时刻



请大家思考下面的问题：

1. 你相信做出的结果是正确的吗
2. 如何保证这个软件的质量

五种上下文方式的比较与组合策略

五种上下文方式对比

上下文方式	优势	劣势	示例
会话上下文	连续、历史回溯	容易被遗忘，太长会超出上下文长度限制	上一轮对话中提到的需求变更
规则上下文	持久、全局	可能不及时、不灵活	项目编码规范、技术栈
自动上下文	便捷、被动	可能"猜"错、不及时	查找函数调用、理解代码结构
手动上下文	精准、主动	依赖用户输入、需要用户对项目内部结构有一定了解	引用特定文件、解释复杂概念
运行时上下文	动态、验证	依赖是否能自动收集到实际运行的信息、需要用户提供验证规则或者测试用例	调试报错、验证修复

推荐组合策略/优先级

1 规则上下文

高优先级基线

先定义再执行，定期更新

2 自动上下文

默认第一步

首先使用自动检索获得全局线索

3 手动上下文

第二步校准

在自动结果不准确时，立即介入

4 运行时上下文

最高优先级收口

进行运行结果验证

5 会话上下文

低优先级长期依赖

用于承接同一任务的连续动作

总结

构建
智能助手的记忆体系

上下文工程的核心，在于系统化地管理智能助手的短期工作记忆（会话上下文）与长期知识库（项目/规则上下文）。

项目是知识的容器

定义了智能助手的认知边界与作用范围

上下文是智能的基石

决定了智能助手在特定任务中的理解深度与执行精度

氛围编程的关键，在于根据任务需求，动态编排并提供最精准的上下文信息。

End

2.氛围编程的关键实践 (2)

—Agentic IDE基于项目的上下文管理